

JDBC



Universidad de Oviedo

Repositorios de Información

Escuela de Ingeniería Informática

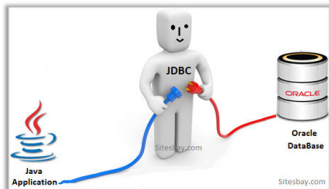
Curso 2026-2027

Section 1

JDBC: Conceptos y características

Definición de JDBC (Java Database Connectivity)

JDBC es una **API estándar de Java** que ofrece a las aplicaciones una forma de acceder a bases de datos relacionales.



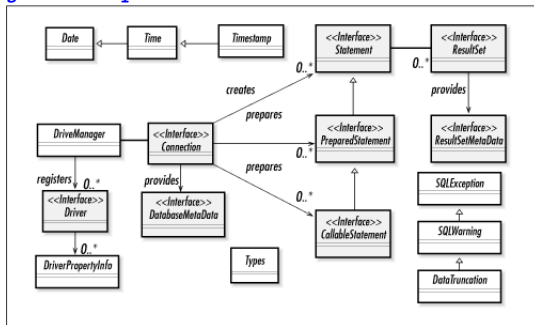
Está compuesta por interfaces y clases Java que permiten:

- **conectar** con una base de datos relacional (o fuente de datos tabular),
- **ejecutar sentencias SQL**,
- **recibir los resultados** o errores y procesarlos.

API JDBC: dos paquetes principales

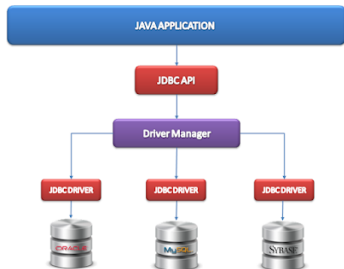
`java.sql` **clases e interfaces** para acceder a la BBDD.

`javax.sql` **características adicionales**



Los proveedores de bases de datos ofrecen un **driver** que implementa las interfaces JDBC básicas, responsables de la comunicación directa con la base de datos.

Componentes de la arquitectura JDBC

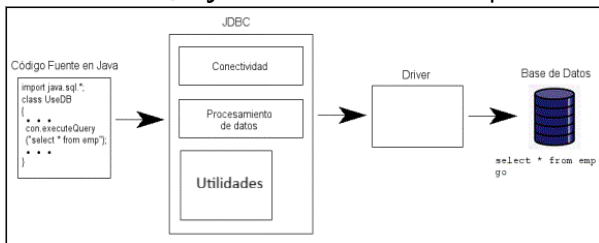


- **Aplicación Java** → invoca métodos de la **API JDBC** para acceder a los datos.
- **API JDBC** → interfaz orientada a objetos para interactuar con bases de datos SQL desde Java.
- **Driver** → implementación de la API para un SGBD específico.
- **Driver Manager** → selecciona el driver adecuado.
- **Origen de datos** → base de datos relacional accesible con SQL.

Controlador o Driver JDBC: Funciones

El controlador JDBC hace posible que el cliente acceda a los datos de la base de datos.

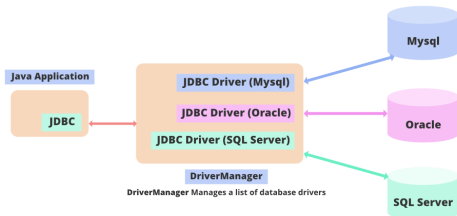
- Establece la **conexión** con la base de datos.
- **Traduce** llamadas del cliente a comandos específicos de la base de datos.
- **Convierte** tipos SQL a clases Java y viceversa.
- Convierte **errores SQL y de conexión** en excepciones Java.



DriverManager: Gestión de controladores

La clase estática DriverManager busca y selecciona el controlador adecuado para establecer la conexión con la base de datos.

- Mantiene una lista de clases Driver disponibles para una aplicación.

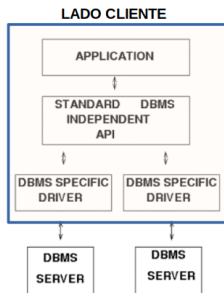


- Cuando se intenta conectar con la base de datos, determina el controlador adecuado, **de forma transparente** al programador.

Desacopla el código cliente del controlador real utilizado. No hay dependencia en tiempo de compilación del controlador específico.

Ventajas de JDBC: **Independencia del SGBD**

- Acceso de clientes mediante una **API uniforme**.
 - JDBC proporciona interfaces y clases estándar como Connection, Statement, ResultSet, etc.
 - El programa cliente utiliza las interfaces.
- **Drivers específicos** de cada SGBD (proporcionados por cada proveedor).
- El **driver implementa** la comunicación cliente - SGBD (conectar, ejecutar sentencias SQL, retornar resultados, etc) a bajo nivel.
- **Conclusión:** el **cliente es independiente** de los detalles de trabajar con un SGBD u otro.



Ventajas de JDBC: **Independencia de la plataforma**

JDBC hereda la independencia de plataforma de Java

una aplicación JDBC puede ejecutarse en distintas plataformas siempre que exista una JVM compatible.

Característica	Responsable
Independencia de la plataforma	Java / JVM
Independencia del SGBD	JDBC + Drivers

Ventajas de JDBC: Mapeo entre tipos SQL y Java

- BBDD **virtual** orientada a objetos sobre la relacional.
- Acceso a la BBDD mediante métodos (API JDBC).
- Conversión automática entre tipos Java y SQL.

Tipo SQL (JDBC)	Tipo Java equivalente	Notas
CHAR, VARCHAR	String	Cadenas de texto
NUMERIC, DECIMAL	java.math.BigDecimal	Para precisión exacta
INTEGER	int / Integer	Entero normal
BIGINT	long / Long	Entero grande
FLOAT, DOUBLE	double	Doble precisión
DATE	java.sql.Date	Solo fecha
TIME	java.sql.Time	Solo hora
TIMESTAMP	java.sql.Timestamp	Fecha y hora

Table: Mapeo de tipos SQL a tipos Java mediante JDBC

Section 2

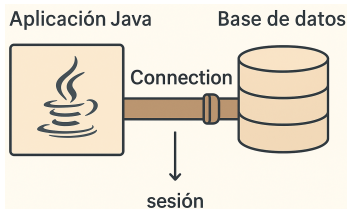
JDBC: Proceso de desarrollo

- *Instalar* drivers JDBC. Más info en lab 01.
- Crear una **conexión** con la BBDD.
- Crear una **sentencia** que contenga una consulta o actualización de la BBDD
- Ejecutar la sentencia
- Recuperar el objeto **resultado**, los errores o las excepciones que ocurriesen.
- Procesar la información contenida en el objeto resultado
- Liberar los recursos empleados: resultado, sentencia y conexión.

Interfaz java.sql.Connection

Crear conexión (básica) con la BBDD

- Representa la **conexión física** entre aplicación y base de datos.
- Implementada en el *driver*.
- No tiene constructor.
- Una aplicación puede tener **múltiples conexiones simultáneas** con **una misma o diferentes base de datos**.
- Una conexión establece una **sesión de trabajo con la base de datos** (se ejecutan sentencias SQL, se reciben resultados, etc).



Establecer una conexión con la base de datos

```
Connection c = DriverManager.getConnection(URL, USER, PASS);
```

Solicita a DriverManager una conexión con una base de datos.

- **USER y PASS:** credenciales para autenticar al usuario en la bbdd.
- **URL:** identifica la bbdd y permite seleccionar un driver adecuado.

Formato general de una URL JDBC:

```
jdbc:[subprotocol]:[host[:port]]:[dbName][;attribute=value]*
```

- **jdbc:** prefijo obligatorio de una URL JDBC.
- **subprotocol:** identifica el SGBD y el driver o protocolo utilizado.
 - **oracle:thin** → Oracle mediante el driver Thin.
 - **hsqldb:hsqldb** → HSQLDB en modo servidor.
 - **hsqldb:mem** → HSQLDB en memoria.
- **host:** servidor donde se encuentra la base de datos.
- **port:** puerto del servidor.
- **dbName:** identifica la base de datos.
- **attribute=value:** atributos opcionales del driver.

Ejercicio

Considera la siguiente información

	Oracle	HSQLDB servidor	HSQLDB memoria
Subprotocol	thin	hsql	mem
Host	156.35.94.98	localhost	—
Puerto	1521	default	—
Nombre BD	desa19	—	test
User	tu id	sa	sa
Password	tu pass	""	""

En cada caso:

- escribe la URL JDBC;
- crea una conexión con la base de datos.

java.sql.Statement

Construcción y ejecución de sentencias

Interfaz Java que permite enviar y ejecutar sentencias SQL en una base de datos.

1. Crear un objeto Statement a partir de una conexión.

```
Statement stmt = connection.createStatement();
```

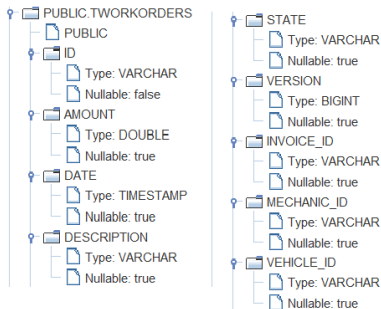
2. Ejecutar sentencias SQL

Method	Returns	Used for
<code>executeQuery(sqlString)</code>	<code>ResultSet</code>	<code>SELECT</code> statement
<code>executeUpdate(sqlString)</code>	<code>int</code> (rows affected)	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , or a DDL
<code>execute(sqlString)</code>	<code>boolean</code> (true if there was a <code>ResultSet</code>)	Any SQL command or commands

3. Obtener el resultado de la ejecución:

dependiendo del tipo de sentencia SQL, la ejecución puede devolver un conjunto de resultados (`ResultSet`) o el número de filas afectadas.

Ejercicio



Escribe y ejecuta sentencias SQL que

- recuperen todas las filas y columnas de la tabla
- todas las filas de la tabla con estado `FINISHED`
- añadan una nueva fila a la tabla

Interfaz java.sql.ResultSet

Recuperación de resultados

ResultSet es un objeto que **representa** los **resultados** de una consulta SQL ejecutada sobre una base de datos.

- Permite **navegar** esos datos **fila por fila y columna por columna**.
- Mantiene un **cursor**, o puntero que apunta a una fila específica.
 - Cuando se crea un ResultSet, el cursor está **antes de la primera fila**, es decir, no apunta a ninguna fila todavía.
 - **Navegación secuencial** fila por fila.
 - **read-only**



Acceso a los datos

1. Acceso a una fila.

- `ResultSet::next()`, retorna boolean que indica si había una fila siguiente a recuperar o no.

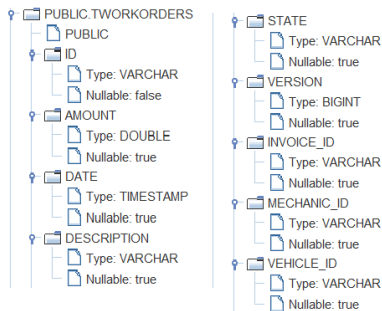
2. Acceso a los **valores de una columna en la fila actual** (cursor) del `ResultSet`.

- Métodos de `ResultSet` que dependen del tipo de la columna: `getInt`, `getString`, `getFloat`, etc.
- Se pueden acceder a las columnas por nombre o posición: `getInt(String fieldName)`, `getString(int columnIndex)` ¹
- El driver convierte los tipos SQL a tipos de datos Java.

Consulta otros métodos en <https://docs.oracle.com/javase/8/docs/api/java/sql/ResultSet.html>

¹Primera columna tiene índice 1, no 0

Ejercicio



Partiendo de las dos consultas anteriores, escribe el código que recorre el resultado e imprima los identificadores y la cantidad a pagar.

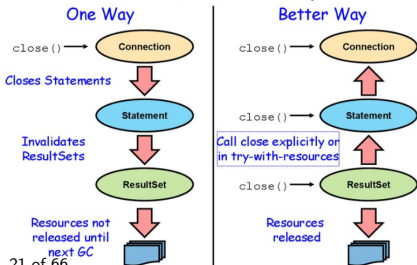
Liberar recursos

Proceso de desarrollo

- Cerrar ResultSet y Statement una vez procesado el resultado.
 - Pueden mantener recursos del SGBD, como cursores.
 - El SGBD puede establecer un límite al número de cursores abiertos.
- Cerrar la conexión cuando haya terminado todo el procesamiento con la base de datos.
 - No es necesario cerrar la conexión después de cada sentencia.

Los recursos deben cerrarse incluso cuando se produce una excepción.

Se recomienda utilizar **try-with-resources**.



Gestión de errores

Proceso de desarrollo

1. `ClassNotFoundException` durante la carga del controlador.
2. `SQLException` durante el acceso a la BBDD.

```
try{
    // all statements including commit
}catch(SQLException e)
{
    // rollback or commit if rollback for a given savepoint
}
finally{
    // close connection or statements or resultsets
}
```

3. Manejar NULL: NULL en SQL es diferente a null de Java.
 - ❑ Evita usar métodos `getXXX( )` que retornan tipos de datos primitivos. Realizan transformaciones que no siempre son válidas.
 - ❑ Usa clases envoltorio (wrapper) para tipos de datos primitivos y comprueba si son null.
 - ❑ Para tipos de datos primitivos usa `wasNull()`.
 - ❑ Proyecto `unit1-manage-nulls`

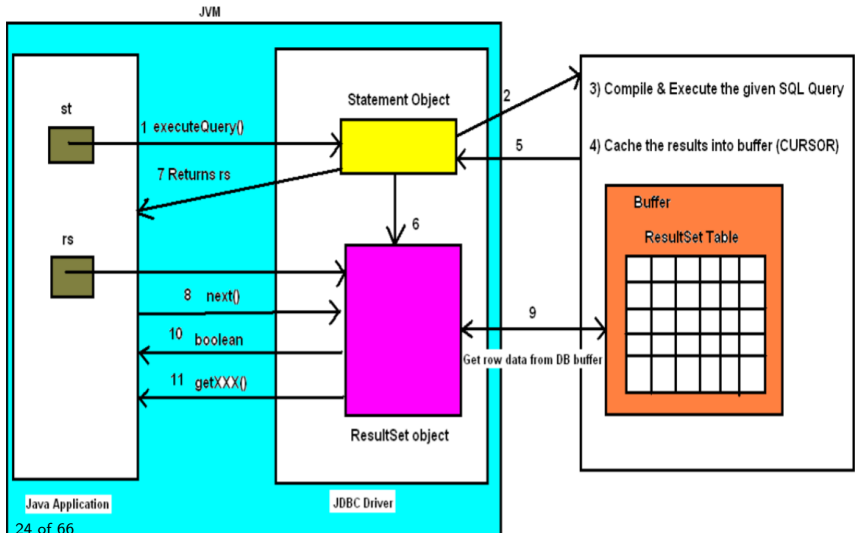
Resumen

Proceso de desarrollo

```
public class MyClass {
    public void doSomething() {
        try (Connection conn = DriverManager.getConnection(URL,
            USERNAME, PASSWORD);
            PreparedStatement stmt = conn.prepareStatement("SELECT_
                a,_b_ FROM _my_table");
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                // Process the result set
                System.out.println("Column_1:_ " + rs.getString("a")
                    );
                System.out.println("Column_2:_ " + rs.getString("b")
                    );
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Resumen

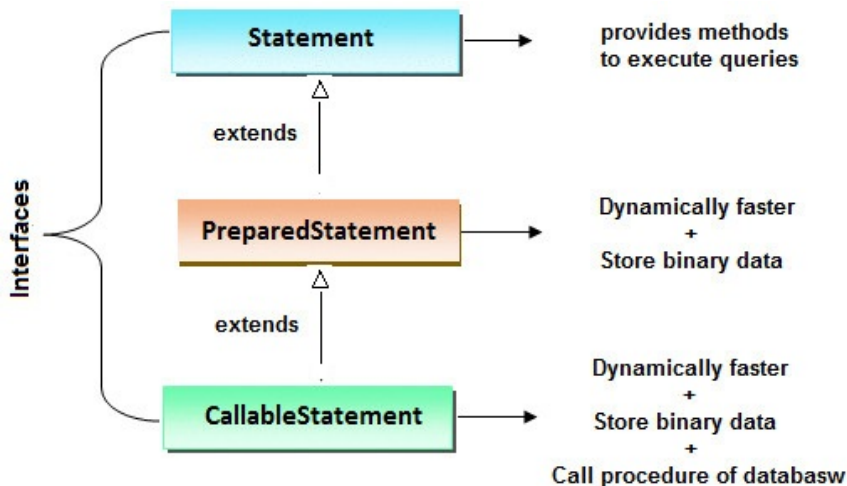
Proceso de desarrollo



Section 3

JDBC: Conceptos Avanzados

Statements



Interfaces java.sql.Statement y java.sql.PreparedStatement

- **Statement:** Se usa para ejecutar sentencias SQL estáticas. Si una misma sentencia se ejecuta repetidas veces, el rendimiento puede ser inferior al de PreparedStatement.
- **PreparedStatement:** Se usa para ejecutar sentencias SQL precompiladas.
 - Escribir la consulta **con parámetros**. Se usa el **marcador (?)**
 - Dar valores a **todos** los parámetros **antes** de ejecutar la sentencia
 - Mejor rendimiento si las consultas se van a ejecutar varias veces.

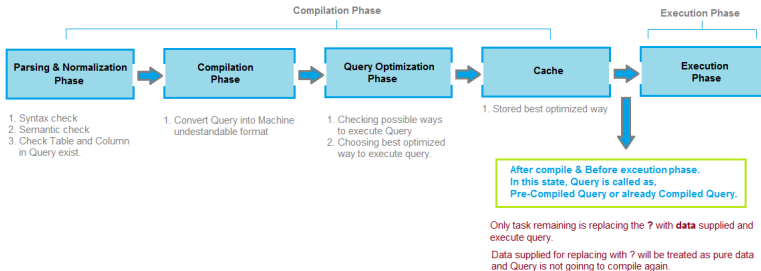
```
Statement stm = con.createStatement();
String updateString = "UPDATE COFFEES SET SALES = 75 " +
    "WHERE COF_NAME LIKE 'Colombian'";
stm.executeUpdate(updateString);
```

```
PreparedStatement updateSales = con.prepareStatement(
    "UPDATE COFFEES SET SALES = ? WHERE COF_NAME LIKE ? ");
updateSales.setInt(1, 75);
updateSales.setString(2, "Colombian");
updateSales.executeUpdate();
```

Realizan lo mismo

Fases de la ejecución de una sentencia

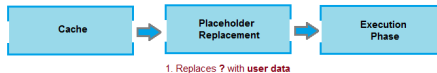
Query Execution Phases



Beauty of PrepareStatement

Query Execution Phases

Beauty of PreparedStatement .



After replacement, Query string formed is not compiled again.

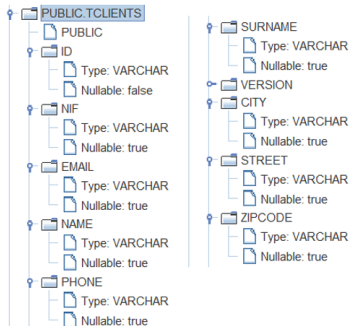
If Query string is not compiled again, it means, If user data contains SQL then also it will not run and will be treated as pure data.

Query stored in Cache is pre-compiled, So it doesn't has to compile again, only replace placeholders and execute query.

Statement vs PreparedStatement

Statement	PreparedStatement
Adecuado para sentencias que se ejecutan de forma ocasional	Adecuado para sentencias que se ejecutan repetidamente
En ejecuciones repetidas, puede tener un coste mayor	Puede ofrecer mejor rendimiento en ejecuciones repetidas
La sentencia SQL totalmente construida antes de ejecutarla	La sentencia SQL se define con parámetros (?)
Puede requerir concatenación para construir SQL dinámico	Facilita el uso de valores dinámicos mediante parámetros (setXXX())
Puede utilizarse para DDL y DML	Puede utilizarse para DDL y DML
	Reduce el riesgo de inyección SQL al separar la sentencia de los valores

Ejercicio



Escribe y ejecuta consultas que retornen nombre y apellidos de clientes de

1. LLANERA.
2. todas las poblaciones de la lista `List<String> towns = {"LLANERA", "OVIEDO", "LUGONES"}` que fueron proporcionadas por el usuario en tiempo de ejecución.

Transacciones

Una transacción es un **conjunto de una o varias operaciones SQL** que se ejecutan como una sólo operación.

- Facturar workorders implica operaciones sobre las tablas TWORKORDERS, TINVOICE.
- Se buscan las workorders a facturar, se comprueba que no estén ya facturadas y que estén terminadas.
- Se marcan (update) como facturadas (no se podrán volver a facturar).
- Se crea una invoice.

Todas las operaciones **deben** realizarse como una única operación.

```

try (Connection conn = DriverManager.getConnection(url, user, password)) {
    String selectSQL = "SELECT * FROM TWORKORDERS WHERE ...";
    try (PreparedStatement selectStmt = conn.prepareStatement(selectSQL)) {
        try (ResultSet rs = selectStmt.executeQuery()) {
            while (rs.next()) {
                // Procesando work order: id
                totalAmount = totalAmount + workorderAmount;
                String updateSQL = "UPDATE TWORKORDERS SET status = ? WHERE id = ?";
                try (PreparedStatement updateStmt = conn.prepareStatement(updateSQL)) {
                    updateStmt.setString(1, "FINISHED");
                    updateStmt.setString(2, id);
                    updateStmt.executeUpdate();
                }
            }
            // Insertar factura asociada
            String insertSQL = "INSERT INTO TINVOICES (...workorder_id, amount) VALUES (?, ?)";
            try (PreparedStatement insertStmt = conn.prepareStatement(insertSQL)) {
                ...
                insertStmt.setString(i, workOrderId);
                insertStmt.setDouble(j, totalAmount);
                insertStmt.executeUpdate();
            }
        }
    }
} catch (SQLException e) {
    ...
}

```

Transacciones en JDBC

- En JDBC, una transacción se ejecuta **dentro de una conexión**.
- Todas las operaciones ejecutadas sobre una conexión forman parte de la misma transacción hasta un `commit()` o `rollback()`.
- Por defecto, JDBC ejecuta **auto-commit** activado:
 - Cada instrucción SQL se confirma automáticamente.
- Para manejar transacciones manualmente:
 - Desactivar auto-commit: `conn.setAutoCommit(false);`
 - Ejecutar operaciones.
 - Confirmar con `conn.commit();`
 - Revertir con `conn.rollback();` en caso de error.

Ejercicio Modificar el código anterior haciendo que todas las sentencias SQL formen parte de una única transacción

Section 5

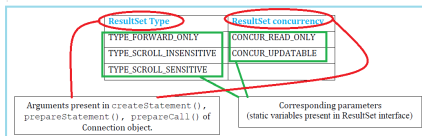
ResultSet mejorado

Capacidades mejoradas para ResultSet

- **sensibilidad**, ResultSet que refleja los cambios realizados por otros procesos en la base de datos después de su creación (del ResultSet).
- **desplazamiento** y **posicionamiento**, el cursor puede moverse en ambas direcciones y se puede colocar en una fila particular
- **actualización**, ResultSet que permite modificar los datos de la base de datos directamente a través del ResultSet, sin necesidad de ejecutar comandos UPDATE o INSERT por separado.

Métodos sobrecargados en el objeto Connection

Statement	<code>createStatement()</code> Creates a Statement object for sending SQL statements to the database.
Statement	<code>createStatement(int resultSetType, int resultSetConcurrency)</code> Creates a Statement object that will generate ResultSet objects with the given type and concurrency.
PreparedStatement	<code>prepareStatement(String sql)</code> Creates a PreparedStatement object for sending parameterized SQL statements to the database.
PreparedStatement	<code>prepareStatement(String sql, int resultSetType, int resultSetConcurrency)</code> Creates a PreparedStatement object that will generate ResultSet objects with the given type and concurrency.



Los parámetros modifican el ResultSet generado aunque pueden generar **sobrecargas** [5]

- `rsType` establece la **navegabilidad**, **posicionamiento** y la **sensibilidad** del ResultSet (capaz de reflejar cambios realizados a la BBDD desde fuera de la aplicación)
- `rsConcurrency` determina si el ResultSet es **read-only** o **modificable**.

rsType: Desplazamiento del cursor y sensibilidad a los cambios

Constante	Descripción
TYPE_FORWARD_ONLY	Navegación secuencial (Sin scroll, sin posicionamiento), insensible a cambios en la BBDD.
TYPE_SCROLL_INSENSITIVE	Scroll, posicionamiento, insensible a cambios en la BBDD (a menos que se ejecute nuevamente la consulta).
TYPE_SCROLL_SENSITIVE	Scroll, posicionamiento, sensible a cambios en la BBDD.

Modificación del objeto ResultSet

Constant	Description
<code>CONCUR_READ_ONLY</code>	El objeto <code>ResultSet</code> no puede ser actualizado (read-only).
<code>CONCUR_UPDATABLE</code>	Se pueden modificar datos en memoria y reflejar esos cambios automáticamente en la base de datos.

Limitaciones del ResultSet

ResultSet mejorado

Para que el ResultSet pueda ser **actualizable** o **sensible a cambios**:

- No se puede usar `SELECT *`.
- La consulta se debe realizar sobre una única tabla, sin joins.

Para que sea **actualizable**, además:

- Para que los `INSERT` puedan realizarse, la consulta debe seleccionar columnas no nulas y sin valores por defecto.
- No pueden seleccionarse columnas agregadas o derivadas de otras (`SUM` o `MAX`), sólo columnas de la tabla.
- Se recomienda comprobar si la **BBDD subyacente admite ResultSet actualizables** o no.

Ejemplos

ResultSet mejorado

Ejecuta los siguientes proyectos (disponibles en el CV) y comprueba qué es lo que sucede y por qué.

1. **ResultSet-Scrollability**: Comprueba la capacidad para **mover el cursor** por el ResultSet y para **modificar el contenido** del ResultSet.
 - Configura parámetros según lo que quieras probar.
 - ¿Puedes insertar² una fila en el ResultSet?
2. **ResultSet-Sensitivity**: Comprueba según la operación **qué cambios son visibles**.
 - Escoge la opción 8 (actualizable + sensible + externo). Ejecuta. Desde SQLDeveloper cambia la **primera y última filas** y continúa.

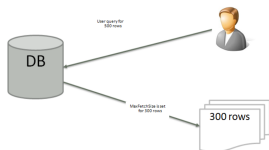
²En páginas posteriores, se detallan las operaciones de la actualización, inserción y borrado de datos en un ResultSet.

Control del fetch size (tamaño de la búsqueda) del ResultSet

ResultSet mejorado

- Los datos devueltos por una consulta no se almacenan en el objeto ResultSet, sino que permanecen en el servidor de BBDD
- ¿ Cuándo se lee una fila del servidor y se cachea en el ResultSet?
- ¿Cómo podemos ejecutar consultas que devuelvan un volumen de datos mucho mayor de lo que se puede guardar en memoria?
- El driver determina el número de filas que se leen a memoria:

Fetch size



```
rs.setFetchSize(25);
```

Las llamadas a `rx.next()` devuelven los datos en caché hasta la fila 26, en cuyo momento el controlador buscará otras 25 filas.

Ejemplos

ResultSet mejorado

■ ResultSet-Sensitivity

- Descomenta la línea 16 de Sensitive.java. Ejecuta de nuevo.
- Elige la opción 8 (actualización + sensible + externo). Ejecuta. Desde SQLDeveloper cambia la **primera y última** fila y continúa.
- En casa, rehacer con ojdbc6 y ojdbc8. Anote las diferencias.

Resumen de la visibilidad de cambios internos y externos

ResultSet mejorado

¿ Puede un objeto `ResultSet` (en la implementación de Oracle JDBC) ver los cambios realizados

- al propio `ResultSet`?
- en la BBDD desde otra transacción, sin ejecutar la consulta otra vez? Lee acerca de *fetch-size* y *window* en [5]

Result Set Type	Can see internal DELETE?	Can see internal DATE?	Can see UP- INSERT?	Can see external DELETE?	Can see external DATE?	Can see UP- INSERT?
forward-only	no	yes	no	no	no	no
scroll-sensitive	yes	yes	no	no	yes	no
scroll-insensitive	yes	yes	no	no	no	no

Realizar una operación INSERT en un ResultSet

La interface ResultSet ofrece métodos para cambiar el contenido del ResultSet pero debe haberse creado updatable.

1. `moveToInsertRow()` coloca el cursor en la *insert-row*, un área que contiene datos para la fila insertada hasta que se copia en la base de datos.
2. `updateXXX()` le permite escribir datos en los campos de la fila
`rs.updateInt(1, integerValue);`
`rs.updateString(2, "stringValue");`
3. `insertRow()` inserta la *insert-row* en el ResultSet y en la tabla. El cambio se hace permanente con la siguiente operación COMMIT de la transacción.

Realizar una operación DELETE en un ResultSet

ResultSet debe ser actualizable.

1. Coloca el cursor en una fila seleccionada usando uno de los métodos de posicionamiento en el objeto ResultSet.
2. Elimina la fila usando `void deleteRow()`. Elimina la fila actual del objeto ResultSet y de la base de datos subyacente; no se puede llamar cuando el cursor está en la fila de inserción.
3. Para asegurarse de que la operación de la fila eliminada se refleje en el conjunto de resultados, debe actualizar el conjunto de resultados con un `ResultSet.close()` y luego crear un nuevo ResultSet con el método `Statement.executeQuery()` .

Realizar una operación UPDATE Result Set

ResultSet debe ser actualizable.

1. Coloca el cursor en una fila seleccionada.
2. Actualice los datos de la columna usando `updateXXX(num/name, type)` que actualiza un objeto de tipo en la columna con número, número o nombre.
3. Cuando haya actualizado las columnas según sea necesario, debe usar `void updateRow()` para actualizar la tabla de la base de datos. No se puede llamar a este método cuando el cursor está en la fila de inserción.
4. Si decide que no desea actualizar la fila, puede llamar al método `void cancelRowUpdates()` en su lugar.

Cuándo usar ResultSet modificable

```
while (rs.next()) {  
    if (rs.getDouble("salary")  
        < 1500) {  
        pstmt.setDouble(1, 1500);  
        pstmt.setInt(2,  
            rs.getInt("id"));  
        pstmt.executeUpdate();  
    }  
}
```

```
while (rs.next()) {  
    if (rs.getDouble("salary")  
        < 1500) {  
        rs.updateDouble("salary", 1500);  
        // la propia fila es modificable  
        rs.updateRow();  
    }  
}
```

¿Qué ventaja tiene?

Principalmente, comodidad y encapsulación: puedes modificar la fila que estás recorriendo sin tener que construir explícitamente otra sentencia UPDATE ni indicar de nuevo la clave de la fila.

Section 6

Conexión avanzada

Conexión en JDBC

Proceso de desarrollo

- Tradicionalmente se utiliza `DriverManager.getConnection()` para obtener una conexión.
- En aplicaciones grandes, puede ser necesario gestionar un gran número de conexiones de forma eficiente.
- **Mejora:** JDBC proporciona la interfaz `DataSource` como mecanismo alternativo para obtener conexiones.
- Un `DataSource` puede proporcionar conexiones directamente o gestionar un **pool de conexiones**.

Interfaz javax.sql.DataSource

DataSource proporciona un mecanismo estándar para obtener conexiones.

```
DataSource ds = ...; // crear y configurar

try (Connection conn = ds.getConnection()) {
    // usar la conexión
}
```

- El DataSource se encarga de proporcionar la conexión.
- La aplicación no necesita utilizar directamente DriverManager.
- La configuración del DataSource depende del entorno.

Obtener DataSource mediante JNDI

```
// Acceder al contexto de nombres
Context ctx = new InitialContext();

// Buscar el DataSource por su nombre lógico
DataSource ds = (DataSource) ctx.lookup("MyDB");

// Obtener una conexión
Connection conn = ds.getConnection();
```

- La aplicación conoce únicamente el **nombre lógico** del recurso.
- JNDI permite localizar el DataSource configurado en el servidor de aplicaciones.
- Una vez obtenido el DataSource, la aplicación utiliza `getConnection()` para obtener conexiones.

Configuración y registro de un DataSource

1. **Crear y configurar** el objeto DataSource estableciendo los parámetros de conexión con la base de datos.

```
OracleDataSource ods = new OracleDataSource();

ods.setDriverType(driverType); // thin
ods.setServerName(serverName); // 156.35.94.98
ods.setDatabaseName(dbName);   // desa19
ods.setPortNumber(portNumber); // 1521
```

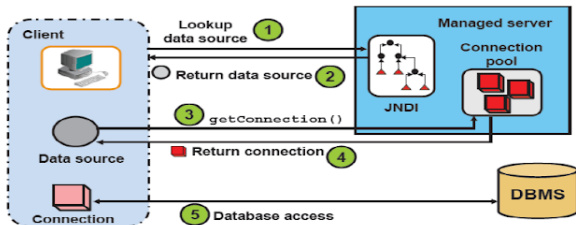
2. **Registrar** el DataSource en un servicio de nombres mediante un **nombre lógico**.

Ver proyecto JNDI-DataSource-Server

Uso del objeto DataSource desplegado

El cliente pueden usar los nombres lógicos para recuperar el objeto DataSource.

Una vez recuperado, se utiliza para obtener conexiones.



Ver proyecto JNDI-DataSource-Client

Para más información, leer [8]

Conexión avanzada

Ventajas de usar DataSource y pool de conexiones

- La interfaz DataSource proporciona una forma **desacoplada** de obtener conexiones:
 - El código cliente no necesita conocer el nombre del controlador ni la URL de conexión.
- Aplicaciones cliente más **portables**.
- Código más **simple** y **fácil de mantener**: los cambios en la configuración de la conexión no requieren modificar el código cliente.

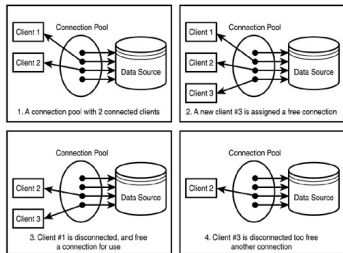
Section 7

Pool de conexiones

Pool de conexiones configurado con DataSource

Un **pool de conexiones** es una **caché de objetos Connection** (**conexiones físicas con bases de datos**) cuyo objetivo es reducir la sobrecarga derivada de conectarse a la bases de datos.

Son **reutilizables** y puede mejorar el **rendimiento**, especialmente en aplicaciones con muchas conexiones.



- Cliente solicita **conexión al DataSource**.
 - Si hay conexión libre en el pool, se devuelve a la aplicación.
 - Si no, se crea una nueva y se devuelve al cliente.
- Cuando la aplicación finaliza su ejecución, **devuelve la conexión al pool**. Dicha conexión estará disponible para una nueva petición.

Alternativas para implementar pools de conexiones

1. **Servidores de aplicaciones:** la mayoría de contenedores implementan su propio pool de conexiones, que las aplicaciones usan mediante DataSource vía JNDI. Permiten configurar grupos de conexiones JDBC para los DataSource.
2. **Pools standalone:** librerías como BasicDataSource o HikariCP, que funcionan sin servidor de aplicaciones.
3. **Librerías de terceros:** útiles si no se está usando un servidor de aplicaciones o un driver JDBC compatible con JDBC 2.0.
 - ❑ OracleConnectionCacheManager (solo Oracle)
 - ❑ c3p0
 - ❑ BoneCP
 - ❑ Jakarta DBCP

Lecturas complementarias (1)

- [1] <https://www.baeldung.com/java-jdbc-loading-drivers>
- [2] <https://northcoder.com/post/class-loaders-service-providers-and/>
- [3] https://en.wikipedia.org/wiki/JDBC_driver
- [4] <https://www.tutorialspoint.com/jdbc/jdbc-db-connections.htm>
- [5] https://docs.oracle.com/cd/B19306_01/java.102/b14355/resltset.htm
- [6] https://docs.oracle.com/cd/A84870_01/doc/java.816/a81354/resltse7.htm

Lecturas complementarias (2)

- [7] Read Oracle Database JDBC Developer's Guide and Reference, chapter 8
- [8] <https://docs.oracle.com/javase/tutorial/jdbc/basics/sqldatasources.html>
- [9] <https://docs.oracle.com/javase/tutorial/jdbc/basics/rowset.html>
- [10] <https://docs.oracle.com/javase/tutorial/jdbc/basics/jdbcrowset.html#creating-jdbcrowset-object>
- [11] <https://examples.javacodegeeks.com/enterprise-java/sql-enterprise-java/javax-sql-rowset-jdbcrowset-example/>

Lecturas complementarias (3)

[12] <https://www.j2eeonline.com/jdbc/module1/intro-to-jdbc.jsp>